PROJECT INTRODUCTION & SCOPE

WHAT THE PROJECT IS ABOUT

Kingdom Quest Adventure Game කියන්නේ fantasy role-playing ක්‍රීඩාවකි. මම මේ ගේම් එකට පහත දේවල් එකතු කරගෙන තියෙනවා:

- **Character types:** විවිධ චරිත වර්ග
- **Enemies:** සතුරන්
- **Weapons:** ආයුධ
- **Magical abilities:** මායා බලයන්
- **Quests:** මෙහෙයුම් / අභියෝග
- **Rewards:** ත්‍යාග
- **Battle system:** සටන් පද්ධතියක්

CORE PURPOSE

Party selection, turn-based combat, accuracy probability, සහ inventory loot වගේ core mechanics model කරලා තමයි අපි මේකේ game development requirements address කරලා තියෙන්නේ.

PROPOSED SOLUTION

Abstract blueprints (Character, Weapon), specialized subclasses, සහ extensible reward interface එකක් යොදාගෙන, core OOP paradigms වලින් තමයි අපි මේක construct කරලා තියෙන්නේ.

VERIFIED SCOPE

Selectable hero classes 3ක් (Warrior, Archer, Wizard), sequential mythological boss quests 5ක්, accuracy hit/miss checks, සහ bounded health logic කියන features මේකට ඇතුළත් කරලා තියෙනවා.

SYSTEM & ENTITY OVERVIEW

PRIMARY ENTITIES (CHARACTERS)

- **Character (Abstract)**
  Central base class for all living participants
- **Warrior**
  Melee fighter (250 HP, sword combat)
- **Archer**
  Ranged hero (100 HP, arrow combat)
- **Wizard**
  Magic caster (80 HP, fireball spell)
- **Enemy**
  Adversaries & bosses holding droppable loot

SECONDARY ENTITIES (ITEMS & OBJECTS)

- **Weapon (Abstract)**
  Base blueprint for all equippable items

- **Sword / Bow / Staff**
  Melee, Ranged & Magic weapon types

- **Inventory**
  List collection of acquired weapons

- **Quest**
  Mission objective, lore, and status flag

- **Reward**
  Loot payload implementing Rewardable

SUPPORTIVE & CONTRACTS

- **Battle (Class)**
  Simulates turn-based combat loop (Attack / Retreat)

- **Main (Class)**
  Program entry point, party setup, 5 quest loop

- **Rewardable (Interface)**
  Behavioral contract: giveReward(Character)

Assumptions

- Quest එක පටන් ගන්න කලින් player හැමතිස්සෙම heroes ලා 3 දෙනෙක්ගෙන් (හැම class එකකින්ම එක්කෙනෙක් විදිහට) party එකක් form කරනවා.
- එක quest එකකදී fight කරන්නේ එක hero කෙනෙක් විතරයි, enemy එක්ක fight කරන්නේ කවුද කියලා player ට select කරන්න පුළුවන්.
- Quests complete වෙන්නේ sequentially, ඒ කියන්නේ enemy ව defeat කළාම quest එක complete වෙලා ඊළඟ එකට move වෙනවා.
- Defeat කරන enemies ලගෙන් weapons drop වෙනවා, ඒ වගේම win කරන hero ඒවා immediately equip කරගන්නවා.
- Health එක maxHealth එක exceed වෙන්නේ නෑ, ඒ වගේම 0 ට සමාන හෝ අඩු වුණොත් defeat වෙනවා.
- Accumulated XP අදාළ threshold එකට ආවම හරි exceed කළාම හරි level-up වෙනවා.
- හැම attack එකකදීම random roll එකක් හරහා accuracy probability එක check කරනවා
- Simplicity එක වෙනුවෙන්, player defeat වුණොත් කෙලින්ම game over වෙනවා.

Requirements

- Predefined character options සහ properties වලින් Warrior කෙනෙක්, Archer කෙනෙක්, සහ Wizard කෙනෙක් බැගින් choose කරන්න system එකෙන් player ට allow කරන්න ඕනේ.
- Unique enemy කෙනෙක්, title එකක්, සහ තමන්ගෙම story description එකක් තියෙන sequential quests 5 ක් system එකේ තියෙන්න ඕනේ.
- System එකේ turn-based combat implement කරන්න ඕනේ. Player තමයි first act කරන්නේ (attack හරි retreat/heal හරි). Attack mechanism එක accuracy එක්ක combine කරලා තියෙන නිසා launch කරන attack එක miss වෙන්නත් පුළුවන්. Enemy තාම alive නම් ඊටපස්සේ same accuracy mechanism එකෙන් enemy ත් attack කරනවා.
- හැම character කෙනෙක්ගෙම HP, XP, level එක, equipped weapons, සහ inventory එක track කරන්න system එකට පුළුවන් වෙන්න ඕනේ.
- Quest එක complete කළාම XP සහ enemy ගේ weapon එක loot එකක් විදිහට award කරන්න system එක හදන්න ඕනේ.
- Active hero defeat වුණොත් හරි, එහෙමත් නැත්නම් whole game එකම complete කළාම හරි game එක end වෙන්න system එකෙන් කටයුතු කරන්න ඕනේ.

Class Design

අපි කලින් discuss කරපු entities වලට necessary stages සහ behaviors carefully implement කරලා තමයි class design part එක අවසන් කරලා තියෙන්නේ. ඒ නිසා, මේ project එකේ total main classes 11 කුත්, supportive classes කීපයකුත් තියෙනවා

Class Descriptions

- **Character (Abstract Class):** Game එක ඇතුළේ ඉන්න හැම living participant entity එකකම foundational attributes සහ behaviors තියෙන්නේ මේ abstract class එකේ. Name, health, level, damage, accuracy, experience points, inventory, equipped weapon, වගේම maximum health කියන properties මේකට ඇතුළත් වෙනවා.
- **Warrior (Class):** මේ class එක Character abstract class එකෙන් inherit වෙනවා. තමන්ගෙම properties එක්ක, sword combat වලට ගැලපෙන විදිහට හදපු specialized attack method එකක් මේකේ implement වෙනවා.
- **Archer (Class):** මේ class එක Character abstract class එක extend කරනවා. Arrow එකක් shoot කරනවා වගේ simulate කරන overridden attack method එකක් මේකෙන් දෙනවා.
- **Wizard (Class):** මේකත් Character abstract class එකේ extension එකක්. Fireball cast එකක් වගේ magical attack එකක් perform කරන්න මේකේ attack method එක override කරලා තියෙනවා.
- **Enemy (Class):** Game එකේ enemies ලව represent කරන්න මේ class එක Character abstract class එක extend කරනවා. මේකේ overridden attack method එකක් තියෙනවා වගේම, enemy ව create කරද්දී optionally weapon එකක් equip කරන්න පුළුවන් වෙන විදිහට overloaded constructors මේකෙන් provide කරනවා.

- **Weapon (Abstract Class):** මේ abstract class එකෙන් තමයි ඔක්කොම weapons වලට අදාළ blueprint එක establish කරන්නේ. Weapon එකේ name එක සහ damage bonus එක මේකෙන් track කරනවා.
- **Sword, Bow,** සහ **Staff (Classes):** මේ classes තුන Weapon abstract class එකෙන් inherit වෙනවා. හැම class එකක්ම abstract getWeaponType() method එක override කරලා, පිළිවෙළින් "Melee", "Ranged", සහ "Magic" විදිහට return කරනවා.
- **Inventory (Class):** ArrayList එකක් පාවිච්චි කරලා, character කෙනෙක්ගේ Weapon objects collection එක manage කරන්නේ මේ class එකෙන්.
- **Rewardable (Interface):** තනි giveReward(Character character) method එකක් specify කරන contract එකක් මේ interface එකෙන් define කරනවා.
- **Reward (Class):** මේ class එකෙන් Rewardable interface එක implement කරනවා. Player කෙනෙක්ට award කරන experience points සහ weapon එක manage කරන්නේ මේකෙන්. ඒ වගේම giveReward method එකට අදාළ logic එකත් මේකෙන් define කරනවා.
- **Quest (Class):** Specific mission එකක progress එක track කරන්නේ මේ class එකෙන්. Title එකක්, description එකක්, completion status එකක්, සහ Reward එකක් මේකේ set up කරනවා.

- **Battle (Class):** Character objects දෙකක් (ඒ කියන්නේ player සහ enemy) අතර තියෙන combat loop එක manage කරන්නේ මේ class එකෙන්. Attacking සහ retreating වලට අදාළ user input handle කරන්නෙත් මේකෙන්.
- **Main (Class):** Application එකේ entry point එක විදිහට act කරන්නේ මේ class එක. Game loop එක initialize කරන එක, characters සහ enemies යොදාගෙන real-world example එකක් execute කරන එක, වගේම quests manage කරන එක මේකෙන් සිද්ධ වෙනවා.

Class Relationships

- Inheritance (IS-A Relationships):

  - Warrior, Archer, Wizard, සහ Enemy කියන හැමෝම abstract Character class එකත් එක්ක IS-A relationship එකක් share කරගන්නවා.
  - ඒ වගේම Sword, Bow, සහ Staff කියන ඔක්කොමත් abstract Weapon class එකත් එක්ක IS-A relationship එකක් share කරගන්නවා.

- Implementation Relationships:

  - Reward class එක මගින් Rewardable interface එක implement කරනවා.

- Composition and Aggregation (HAS-A Relationships):

  - Character class එකට Inventory class එක සහ Weapon class එක කියන දෙකත් එක්කම HAS-A relationship එකක් තියෙනවා.
  - Weapons list එකක store කරන නිසා, Inventory class එකට Weapon class එකත් එක්ක HAS-A relationship එකක් තියෙනවා.
  - Quest class එකට Reward class එකත් එක්ක HAS-A relationship එකක් තියෙනවා.

- Drop වෙන item එක store කරගන්න, Reward class එකට Weapon class එකත් එක්ක HAS-A relationship එකක් තියෙනවා.
- Player සහ enemy variables වලට පාවිච්චි කරන නිසා, Battle class එකට Character class එකත් එක්ක HAS-A relationship එකක් තියෙනවා.

Program Implementation

- Game Initialization and Character Management

Game එක initialize වෙන්නේ central controller එක විදිහට act කරන Main class එක ඇතුළේ. User input capture කරගන්න මේ program එක Scanner object එකක් පාවිච්චි කරනවා. ඒ හරහා Warrior, Archer, සහ Wizard ලගේ specific variations select කරලා තමන්ගේ party එක assemble කරන්න කියලා player ට prompt කරනවා. මේ හැම selection එකකදීම, name, damage, සහ accuracy වලට අදාළ predefined values එක්ක ඒ ඒ character subclass එක initialize වෙනවා.

- Quest and Enemy Sequencing

Main class එකේ තියෙන parallel arrays පාවිච්චි කරලා තමයි අපි application එකේ game's progression එක structure කරලා තියෙන්නේ:

•Progressively difficult bosses ලා පස්දෙනෙක් එක්ක Enemy[] array එකක් අපි instantiate කරලා තියෙනවා. මේ සමහරක් bosses ලව create කරද්දීම weapons (උදාහරණයක් විදිහට Sword හරි Staff එකක් හරි) equip කරන්න overloaded constructors පාවිච්චි කරලා තියෙනවා.

•හැම enemy කෙනෙක්ටම අදාළ වෙන questTitles සහ questDescriptions කියන narrative details තියාගන්නේ String[] arrays දෙකක් ඇතුළේ.

•ඊටපස්සේ for loop එකක් හරහා මේ arrays iterate කරලා, හැම quest එකකටම අදාළ story context එක player ට present කරනවා. ඒ වගේම current enemy එක්ක fight කරන්න, කලින් assemble කරගත්ත heroes ලා තුන්දෙනාගෙන් එක්කෙනෙක්ව select කරන්න කියලත් player ගෙන් අහනවා.

- Battle Simulation Module

- Player සහ enemy අතර turn-based combat එක manage කරන core gameplay loop එක අන්තර්ගත වෙන්නේ Battle class එක ඇතුළේ:
- player.isAlive() සහ enemy.isAlive() කියන conditions දෙකම true වෙනකම් continuously run වෙන while loop එකක් startBattle() method එකේ පාවිච්චි කරලා තියෙනවා.
- Player ගේ turn එකේදී, Attack ද නැත්නම් Retreat ද කියලා action එකක් choose කරන්න Scanner එක හරහා prompt කරනවා.
- "Retreat" තේරුවොත් Character class එකේ තියෙන heal(10) method එක invoke වෙලා, character ගේ maximum limit එක වෙනකම් health එක restore වෙනවා.
- "Attack" තේරුවොත් අදාළ specific character class එකේ (Warrior, Archer, හරි Wizard හරි) overridden attack() method එක trigger වෙනවා. මේ method එකෙන් Math.random() පාවිච්චි කරලා random number එකක් generate කරලා, ඒක character ගේ accuracy එකත් එක්ක compare කරලා බලනවා attack එක land වෙනවද, miss වෙනවද කියලා.
- Player ගේ turn එකෙන් පස්සේ enemy survive වෙලා හිටියොත්, player ව target කරලා තමන්ගේම attack() method එක call කරලා enemy immediately counter attack කරනවා.

- Inventory and Reward Module

  - Battle class එකේදී enemy ව successfully defeat කළාම, gainXp() method එක හරහා player ට base XP 30ක් award කරනවා. ඇති තරම් XP accumulate වුණොත් character ව level up කරන logic එකත් මේ method එකේම තියෙනවා. Battle එකට පස්සේ further progression එක handle කරන්නේ Main class එකෙන්:
  - getEquippedWeapon() පාවිච්චි කරලා enemy ගේ equipped weapon එක extract කරගන්නවා.
  - Bonus XP සහ drop වුණු weapon එක යොදාගෙන අලුත් Reward object එකක් instantiate කරලා, ඒක අලුත් Quest object එකකට pass කරනවා.
  - ඊටපස්සේ Quest එක completed කියලා mark කරලා Reward logic එක execute කරනවා. එතනදි Inventory module එක පාවිච්චි කරලා අලුත් Weapon එක player ගේ internal list එකට add කරනවා වගේම, future battles වලදී total damage output එක increase කරගන්න ඒ weapon එක equip කරනවා.

- Game Over and Victory Handling

    - Battle loop එක අවසන් වුණාට පස්සේ player ගේ health state එක monitor කරන්නේ Main class එකෙන්.
    - !player.isAlive() true වුණොත්, program එකෙන් Game Over message එකක් print කරලා, break statement එක පාවිච්චි කරලා main loop එකෙන් exit වෙලා game එක halt කරනවා.
    - හැබැයි player මේ quest loop එකේ iterations පහම survive කළොත්, game එක successfully complete කළා කියලා පෙන්නන්න final victory message එකක් print කරනවා.